

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM



BÁO CÁO CUỐI KỲ

GV HƯỚNG DẪN: TS. Lê Văn Tuấn

SINH VIÊN THỰC HIỆN:

22520231 : Phạm Tấn Đạt

22520271: Nguyễn Thành Đức

TP. HỒ CHÍ MINH, NĂM 2025

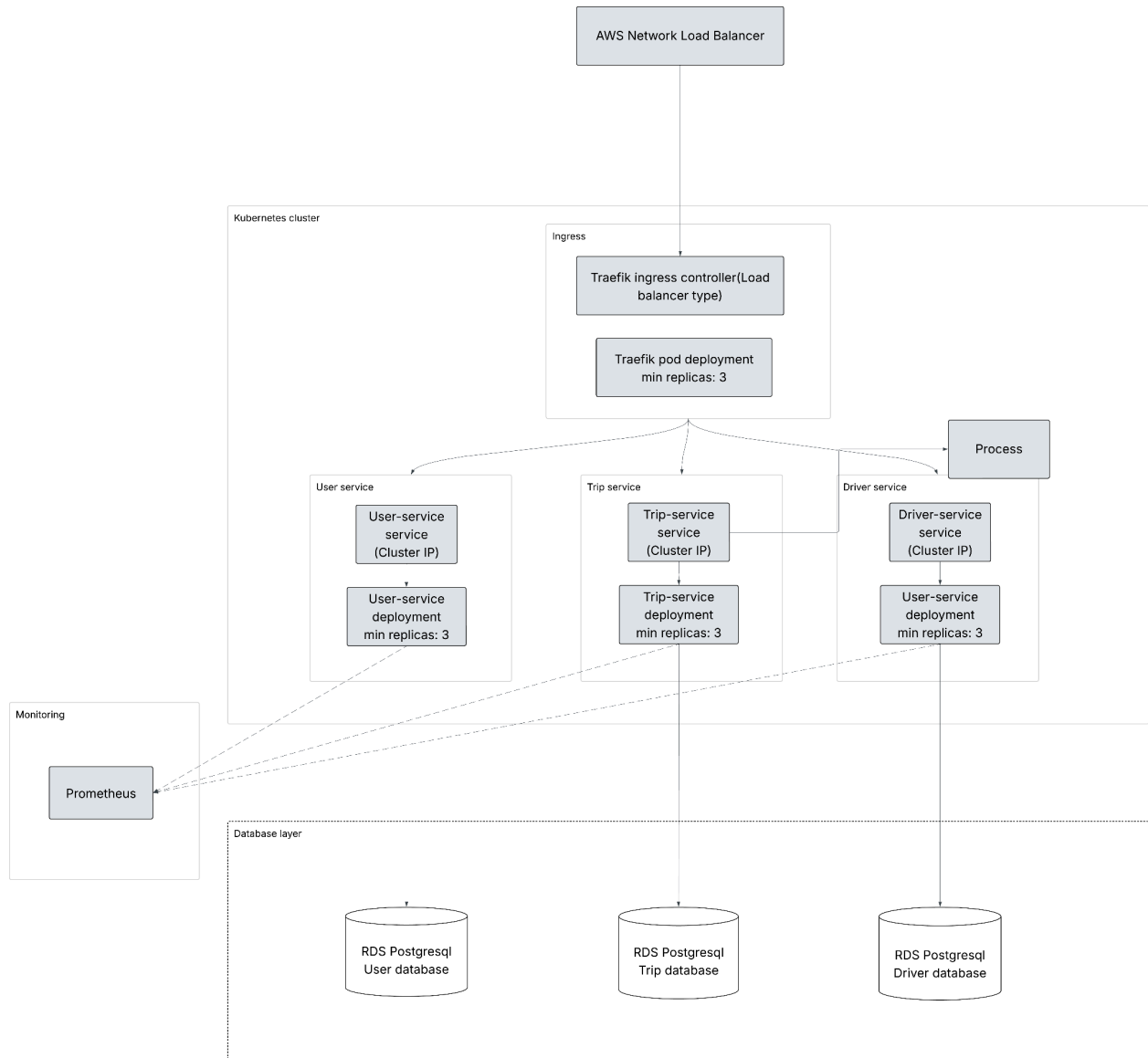
MỤC LỤC

MỤC LỤC.....	1
I. Kiến Trúc Hệ Thống.....	3
1. Sơ đồ kiến trúc.....	3
2. Phân tích các thành phần trong kiến trúc tổng quan.....	3
3. Sơ đồ triển khai hạ tầng trên cloud.....	5
II. Phân tích module chuyên sâu:	
Reliability & High Availability.....	6
1. Mục tiêu tổng quát.....	6
III. Tổng hợp Các quyết định thiết kế và Trade-off.....	6
1. Các công việc đã thực hiện để đáp ứng High Availability.....	6
2. Phân tích trade-off.....	6
IV. Thách thức và bài học kinh nghiệm.....	7
V. Kết quả và hướng phát triển.....	8

BẢNG PHÂN CÔNG CÔNG VIỆC	
Phạm Tấn Đạt - 22520231	Nguyễn Thành Đức - 22520271
- Tham gia xây dựng kiến trúc - Xây dựng kiến trúc hạ tầng - Xây dựng user-service - Xây dựng IaC terraform - Setup k8s, helm - Thực hiện triển khai lên AWS	- Tham gia xây dựng kiến trúc - Xây dựng trip-service, driver-service - Setup docker-file, docker-compose - Viết tài liệu báo cáo.

I. Kiến Trúc Hệ Thống

1. Sơ đồ kiến trúc



2. Phân tích các thành phần trong kiến trúc tổng quan

- Network Load balancer (AWS NLB):

- Đóng vai trò là đầu vào đầu tiên của hệ thống.
- Chuyển tiếp các request đến các ingress-controller trong cluster giúp phân tải cho ingress-controller.

- Được quản lý bởi các cloud-provider nên có High Availability, có thể triển khai ở multi-az giúp tăng availability

- Ingress controller (Traefik):

- Là thành phần chuyển tiếp request đến các service.
- Có khả năng chuyển tiếp request đến đúng service bằng url-prefix.
Triển khai nhiều pod ở nhiều az giúp tăng availability.
- Tự động khai báo instance với Network Load Balancer, giúp có khả năng tự động khôi phục, tạo pod mới khi có lỗi hoặc cần scale.

- Các service (K8s Service, Deployment, Pod):

- Chịu trách nhiệm xử lý business-logic của hệ thống.
- Được triển khai bởi 3 lớp chính
 - +Service: tự động route request tới pod có thể xử lý, đóng vai trò như 1 Load balancer ở mức độ service.
 - +Deployment: chịu trách nhiệm quản lý các pod, khôi phục, chạy mới pod khi có pod chết hoặc cần scale.
 - +Pod: là đơn vị nhỏ nhất, chạy các container.

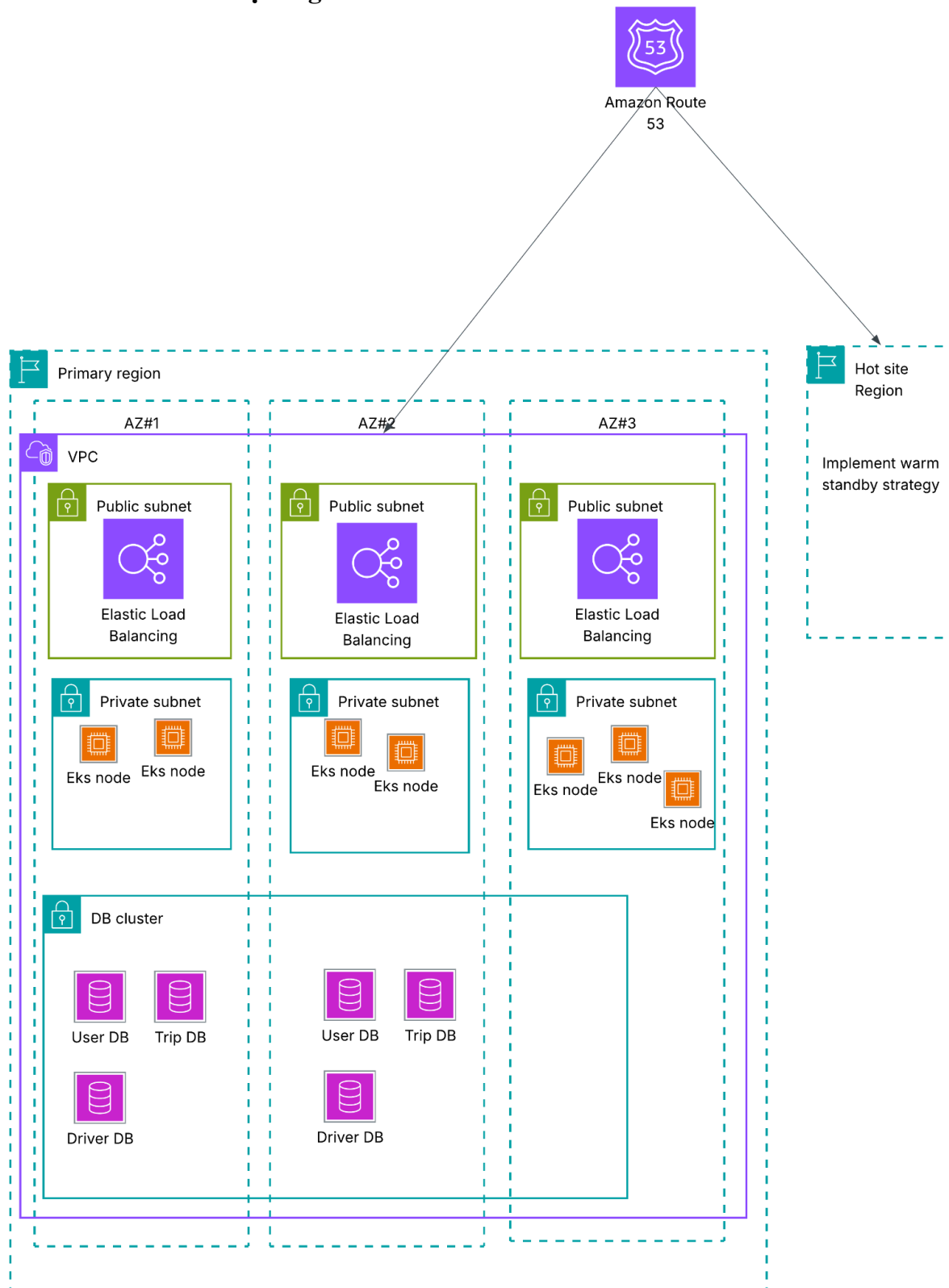
- Database:

- Triển khai các replicas multi-az giúp tăng availability.

- Monitoring (Prometheus):

- Giúp giám sát hệ thống, phát hiện lỗi

3. Sơ đồ triển khai hạ tầng trên cloud



II. Phân tích module chuyên sâu: Reliability & High Availability

1. Mục tiêu tổng quát

- Đảm bảo hệ thống có khả năng hoạt động liên tục ngay cả khi xảy ra sự cố (hardware failure, network failure, region outage,...).
- Xây dựng hệ thống self-healing, có khả năng failover tự động.
- Xây dựng hệ thống có khả năng Disaster recovery

III. Tổng hợp Các quyết định thiết kế và Trade-off

1. Các công việc đã thực hiện để đáp ứng High Availability

- Triển khai multi-instance, ,multi-az:

- Các service, ingress-controller đều được triển khai nhiều instance, ở nhiều az khác nhau nhằm đảm bảo HA.
- Traffic đến các instances đều được xử lý bởi các thành phần đóng vai trò như load balancer (NLB, k8s service LB)
- Khi 1 hoặc nhiều instance(pod) gặp lỗi, các thành phần chịu trách nhiệm sẽ tạo mới đảm bảo hệ thống liên tục hoạt động.
- Sử dụng managed database của AWS, với các multi-az replicas đảm bảo HA ở database layer

- Disaster recovery:

- Triển khai 1 hot-site theo chiến lược warm-standby giúp hệ thống có khả năng DR
- Sử dụng Aurora database của AWS với khả năng sao lưu với **RPO rất thấp (< 1 giây)** và failover tự động.
- **RTO ~ 30s đến vài phút** nhờ các service đã hoạt động ở hot-site, chỉ cần chờ route 53 failover và chuyển hướng traffic.

2. Phân tích trade-off

Cost:

- Sử dụng các dịch vụ managed database như Aurora với các multi-az replicas khiến chi phí tăng cao. Bù lại có HA, backup giữa primary với hote site dễ dàng hơn không cần thực hiện thủ công, RPO thấp do Aurora quyết định.

- Hot-site theo chiến lược warm-standby cũng tiêu tốn nhiều chi phí, bù lại đáp ứng RTO thấp, không cần khởi động các service khi DR.

Độ phức tạp hệ thống:

- Sử dụng K8s deployment với multi-az pods giúp tăng HA, đổi lại phức tạp trong triển khai và vận hành.

Tóm lại: bên trên là những đánh đổi mà hệ thống đã chấp nhận để đáp ứng Reliability & High Availability của hệ thống, đảm bảo các nghiệp vụ quan trọng của 1 ứng dụng đặt xe có yêu cầu cao về availability không bị gián đoạn.

IV. Thách thức và bài học kinh nghiệm

- Trong quá trình giải quyết bài toán, tìm giải pháp và thực hiện bọn em gặp nhiều thách thức:

- Kiến thức về các công nghệ mới khá nhiều, việc làm quen và sử dụng các công nghệ tốn nhiều thời gian.
Chưa có kinh nghiệm về Cost optimization khi sử dụng các dịch vụ của các cloud provider, dẫn đến lãng phí chi phí cho hạ tầng.
- Các giải pháp trong nghiệp vụ của ứng dụng cũng cần nhiều sự tìm hiểu, ví dụ như bài toán đặt xe, tìm tài xế khó để đưa ra giải pháp tối ưu

- Qua đồ án bọn em rút ra được nhiều bài học:

- Cần tìm hiểu kỹ, cân đối giữa việc sử dụng các managed-service và tự xử lý các công việc cần thiết.
- Phải hiểu ra yêu cầu bài toán nghiệp vụ, từ đó đưa ra lựa chọn phù hợp với từng chức năng nghiệp vụ.
- Tối ưu, cân đối giữa các yêu cầu về performance, availability, cost,...
- Kiến thức tổng quan về các công nghệ đã sử dụng trong hệ thống.

V. Kết quả và hướng phát triển

Qua đồ án bọn em đã có kiến thức cơ bản xây dựng 1 hệ thống Reliable, High Availability. Tìm hiểu đưa ra giải pháp và học hỏi về các công nghệ nhằm đáp ứng yêu cầu. Bên cạnh đó bọn em có các đề xuất để phát triển thêm trong tương lai như:

- Xây dựng Automation: thêm các pipeline CI-CD
- Tối ưu chi phí
- Tìm hiểu, tối ưu performance của hệ thống.
Nâng cao các yêu cầu về security.